AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_HWSW_task_5 Version : 1	

HW/SW PROTOTYP AV WATCHDOG TIMER SYSTEM

Darwin Rengifo

2019-05-16

Sammanfattning:
Denna rapport beskriver en förstudie och planering för Watchdog Timer projektet med användningen av en Wathdog Timer system och Qsys-verktyget för att integrerar hårdvaran och mjukvaran interface genom läsningen och skrivningen på Avalon bus. Det är en enhet som måste bli triggad (skriven till) cykliskt inom en viss tid annars utlöses en utgång, som i detta fall ska ge en puls in på reset_n. Denna rapport innehåller aktivitets- och tidplan, projektkostnad, kravspecifikations, projektarkitektur och beskrivning av utvecklade komponenter.

Innehåll

1. INLEDNING 3
2. KRAVSPECIFIKATION FRÅN KUND 4
3. PROJEKPLAN 6
4. HÅRDVARAN ARKITEKTUR FÖR INGÅENDE KOMPONENTER. 7
   4.1 Komponenter 7
   4.1.1 WATCHDOG_TIMER_SYSTEM 7
   4.1.2 wdt_PIO 7
   4.1.3 wdt_KICKER_N 7
   4.1.4 Timer_hw_IP 7
   4.1.4.1 Timer_top 8
   4.1.5 PLL 8
   4.1.6 System ID 8
   4.1.7 On-Chip RAM 8
   4.1.8 JTAG UART 8
   4.1.9 SDRAM Controller 8
5. KONSTRUKTIONS BESKRIVNING 9
   5.1 Systemarkitektur 9
   5.2 Delsystem 11
   5.2.1 Toppnivån 11
   5.2.1.1 Systemarkitektur 11
   5.2.2 Watchdog Timer – komponent 12
   5.2.2.1 Systemarkitektur 12
   5.2.3 Bcd konverter - komponent 13
   5.2.3.1 Systemarkitektur 13
   5.2.4 Sju_seg_displayer - komponent 14
   5.2.4.1 Systemarkitektur 14
   5.3 I/O pinnar 15
6. VALIDERING KONSTRUKTIONEN 15
   6.1 Testprotokoll för validering 15
7. MJUKVARAN ARKITEKTUR 16

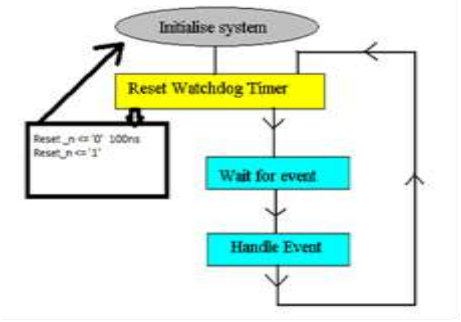
7.1 Minneskarta 16
7.2 Drivrutiner 16
7.2.1 Timer-drivrutiner 16
7.3 C koden 17

1. INLEDNING

Denna rapport beskriver hur man bygger ett Watchdog timer system som efter 500ms ska den räkna upp en 7 segments displayer med ett och ge ut en puls på ”Restart” till ”Watchdog Timer”. När programmet kommer i feltillståndet ska 8 röda lysdioder tändas. Det kommer att validerats på ett lämpligt FPGA-kort.

2. KRAVSPECIFIKATION FRÅN KUND

Tabell 1. Kravspecifikation från kund

Krav	Beskrivning	Utfört
		Ja/nej
001	Rapportnamn: Demonstrationssystem för att öka tillgängligheten till en kaffe maskin. Projekt namn: Watchdog_Timer	ja
	Funktion	
002	Konstruera ett system med en komponent utanför datorn som kallas ”Watchdog_Timer” som ska triggas minst varje sekund av en signal som kallas ”Restart” (kan också kallas kick_the_dog” eller ”feed_the_dog”). Skapa en kontinuerlig puls från C-programmet med en periodtid som är kortare än en sekund när systemet fungerar (kick_the_dog*). Om signalen ”Restart” (se figur 2) är längre än 1 sekund, ska hela systemet återställas – ”reset”-aktiveras två klockcykler.  **Figur 2: Exempel HW arkitektur av Watchdog Timer**	ja
	Funktionskrav	
003	Skriv ett SW-program som exekveras var 500ms i en loop. Efter varje loop ska den räkna upp en 7 segments displayer med ett och ge ut en puls på ”Restart” till ”Watchdog Timer”, se nästa figur (kan också kallas kick_the_dog” eller ”feed_the_dog”).	ja
	 **Figur 1: Exempel på SW architectur**	ja
003A	KEY[0] är ”häng” tryckknapp (vilken skapar ett fel i C-programmet). Om ”häng”-knappen trycks ner ska detta genererar ett feltillstånd som gör att C-programmet går in i en oändlig loop (hänger sig eller ”hang up”). Programmet kan inte komma ur loopen om inte processorn resettas. När programmet kommer i feltillståndet ska 8 röda lysdioder tändas.	ja
003B	SW[0] ska användas för att slå på och av ”Watchdog_Timer”. Om den är avstängd ska ”Watchdog_Timer” inte påverka systemet.	ja
004	Följ VHDL- och C-reglerna vilka gäller för TEIS	ja

005	En kort demonstrationsfilm som visar att systemet fungerar (max 1 min). Inga kvalitetskrav på filmen.	ja
006	Rapporten ska också innehålla kostnaden för konsultuppdraget. Företaget konstruktören jobbar åt fakturerar 600 SEK/timme för projektet. Totalkostnad= antal timmar * 600 SEK.	ja
Leveranskrav		
007	Leveransen ska ske till plattformen Itslearning. Leveransen ska vara en kort standardrapport samt det arkiverade Quartusprojektet, Timer IP komponenten och software. Namnet på filen ska vara ”fornamn_efternamn_HWSW_task_4” (komprimerad). Sista leveransdag se kursschema (för VG).	ja

3. PROJEKPLAN

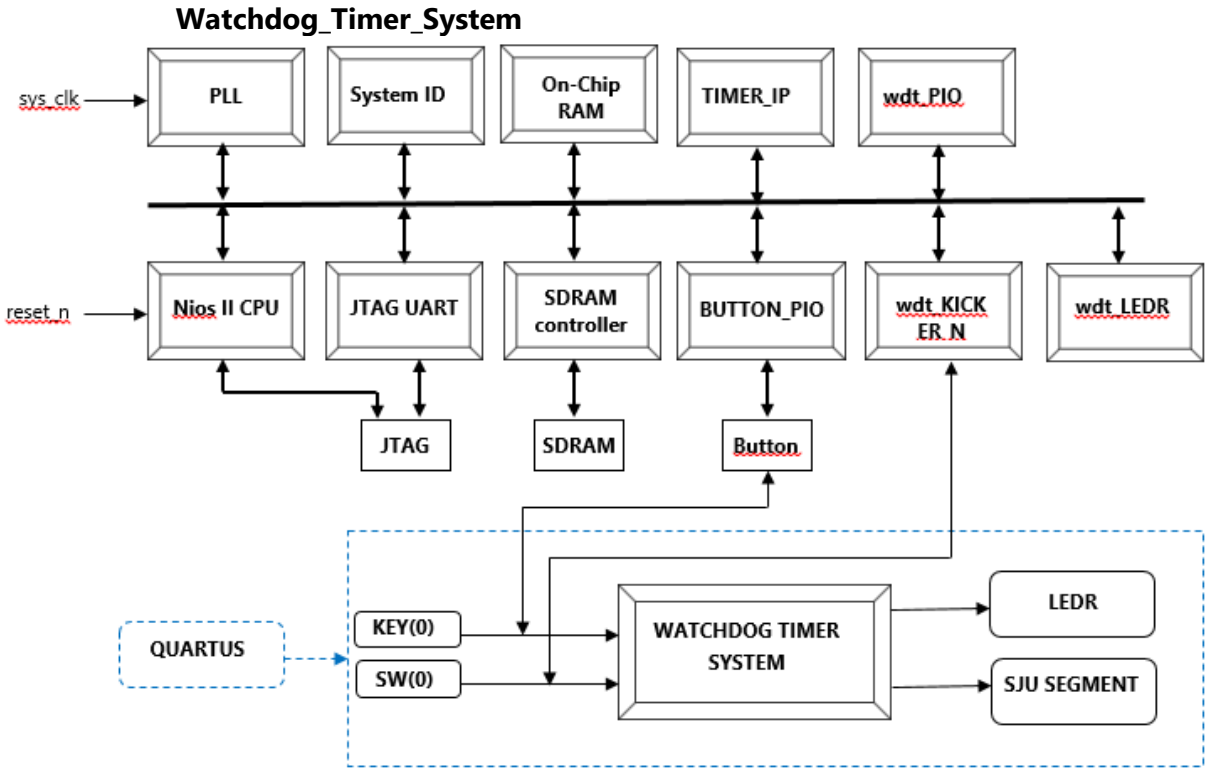
Aktivitets - och tidplan för ingenjörsjobbet kan ses i Tabell 2. Totalt uppskattas ingenjörsjobbet ta 28 timmar/600kr(timme), vilket ger en lönekostnad på 16 800 SEK.

Tabell 2. Aktivitets- och tidsplan för ingenjörsjobbet

Aktivitet	**Veckor**				
	18	**19**	**20**	**Timmar(h)**	**Kostnad(kr)**
Kravspec./förstudie	5			5	3 000
Testprotokoll/ Drivrutiner	1	1	1	3	1 800
Arkitekturbeskrivning	3	3		6	3 600
Programmering av visuellt gränssnitt	4	3	1	7	4 200
Verifiering		1	1	2	1 200
Validering			1	1	6 00
Slutrapport		2	2	4	2 400
Summa tid	13	10	6	28	16 800

4. HÅRDVARAN ARKITEKTUR FÖR INGÅENDE KOMPONENTER.

Systemarkitekturen som FPGA:n programmerats med är en enkel bussarkitektur för att läsa och skriver från IP komponenten med en mikroprocessor av typen Nios II som styr, se Figur 1.



Figur 1. Systemarkitektur, blockschema Watchdog_Timer_System

4.1 Komponenter

I projektet ska flera komponenter utvecklas. En toppfilen komponent som instansierar qsys-interface, en för att skapa avalon bus interface och de andra för att skapar utgången interface av data till skärmen.

4.1.1 WATCHDOG_TIMER_SYSTEM

Filen som ska hanterar Watchdog Timer System.

4.1.2 wdt_PIO

Det skickar en bit signal('0' eller '1') till Nios II när SW(0)=Watchdog Timer är enable/disable.

4.1.3 wdt_KICKER_N

Den signalen "kick_the_dog" Watchdog Timer när det fungerar systemet och när systemet få ett feltillstånd den signal det kommer att återställas hela systemet.

4.1.4 Timer_hw_IP

Filen som skapar Avalon bus interface av läsning/skrivning även instansierar komponent som använder för att ändrar hastighet frekvensen för att visualisering av data på skärmen.

4.1.4.1 Timer_top

Kontrollerar sekvensen av reset-stop-start-read på hårdvaran.

4.1.5 PLL

En PLL finns tillgänglig i systemet för att distribuera varianter av systemklockan på 50 MHz till delar av systemet.

4.1.6 System ID

En komponent som innehåller information om hårdvaruversion.

4.1.7 On-Chip RAM

Intern styrenhet för SDRAM-minnet.

4.1.8 JTAG UART

Kommunikationsgränssnitt mellan USB Blaster och data/instruktionsbus. Används för kommunikation mellan PC och CPU för programmering och debuggning.

4.1.9 SDRAM Controller

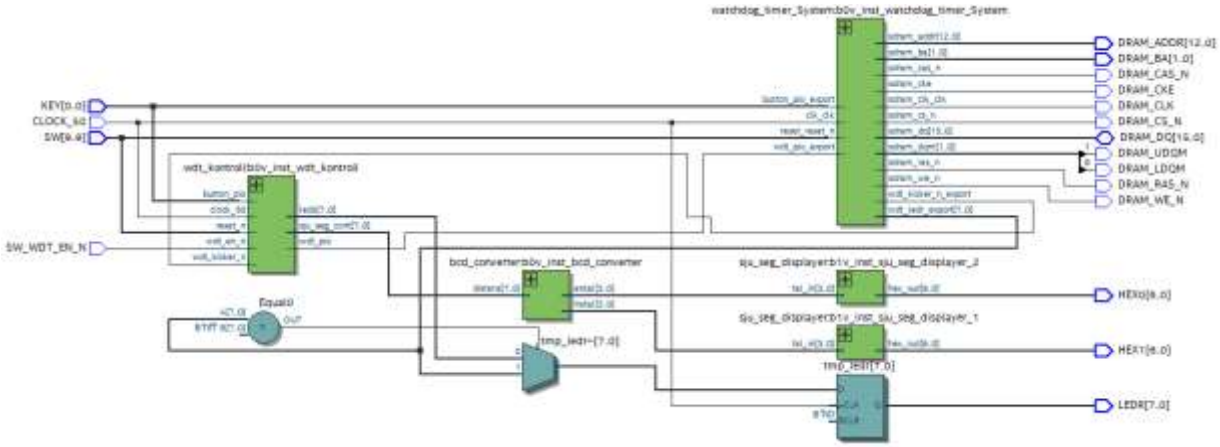
Extern styrenhet för SDRAM-minnet.

5. KONSTRUKTIONS BESKRIVNING

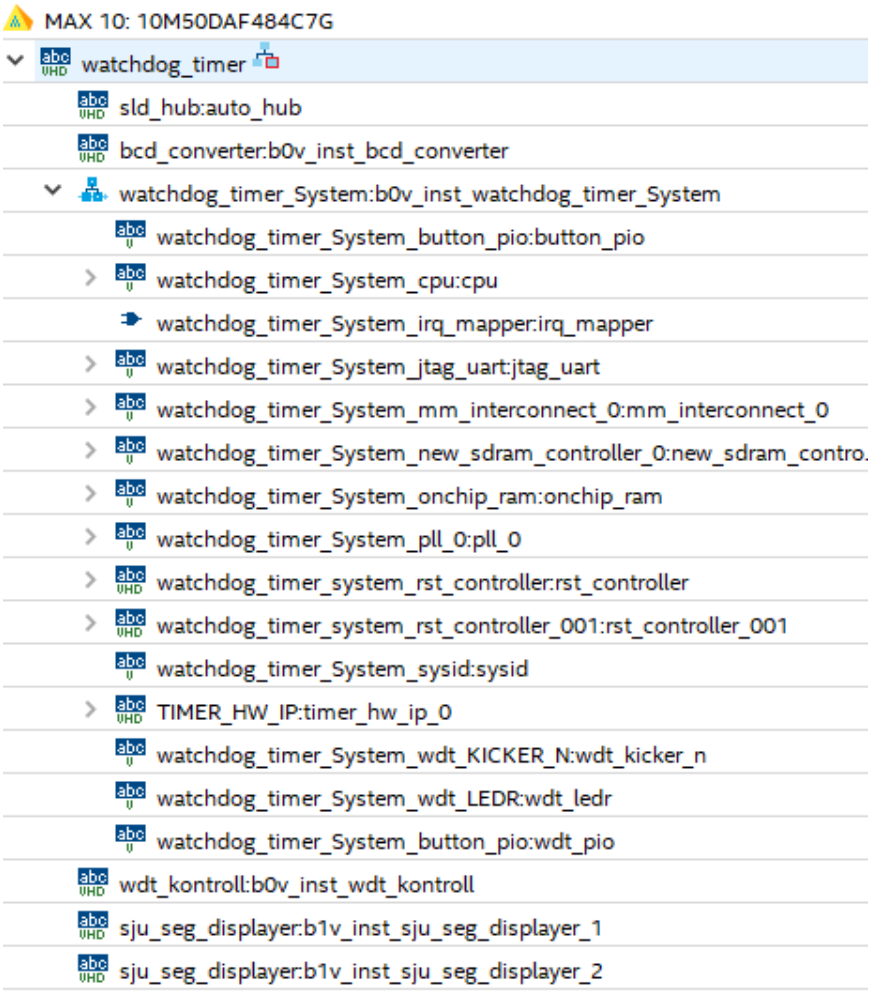
Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet. DE10-Lite-kortet innehåller en sex 7-segment display för att se nummer och tio lysdioder som kan kontrollera efter behov.

5.1 Systemarkitektur

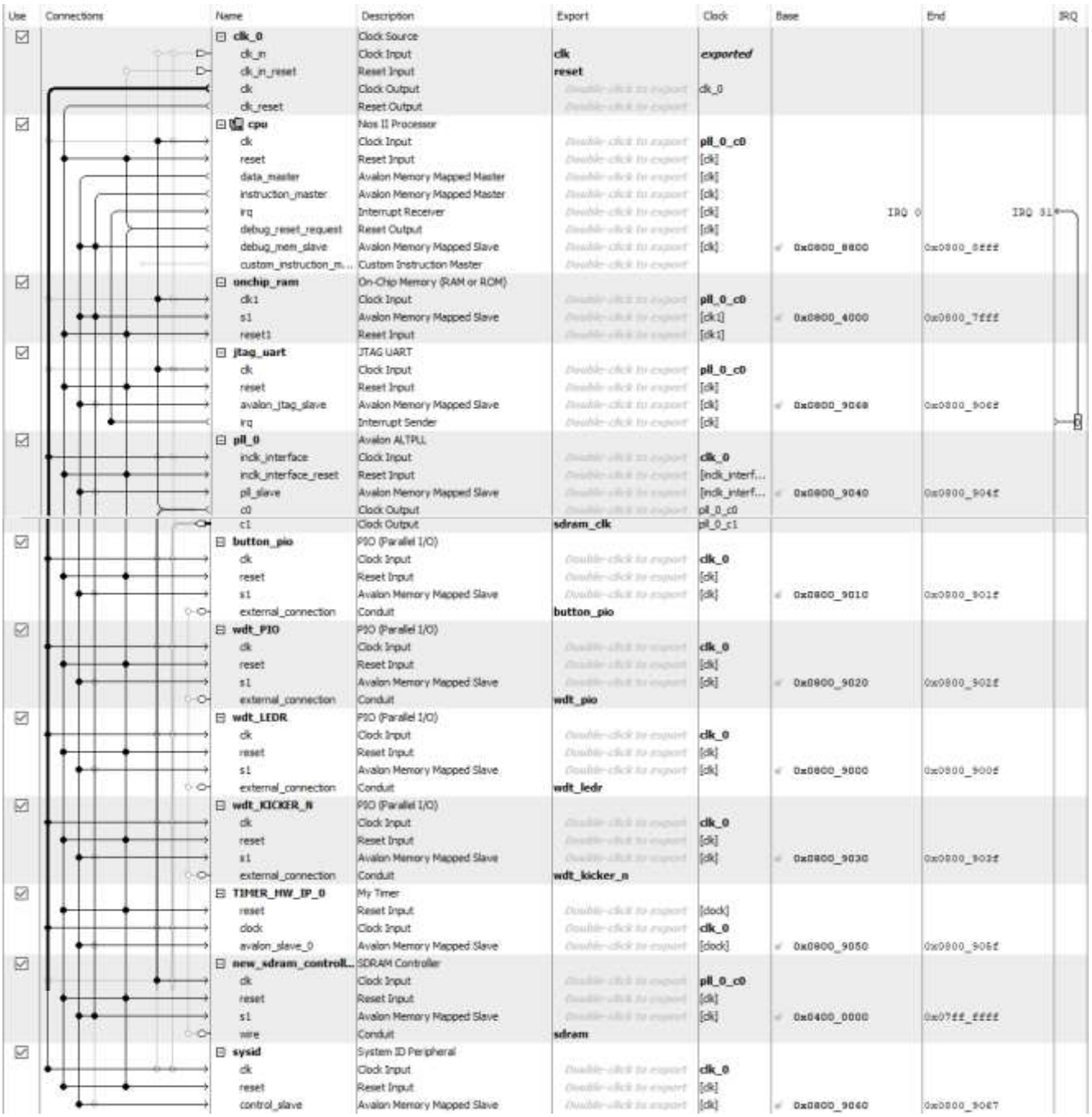
TOPPNIVÅN arkitektur visas i Figur 2, i figur 3 visar komponenthierarki och i figur 4 visar QSYS interface.



Figur 2. Systemarkitekturen för WATCHDOG_TIMER



Figur 3. Komponenthierarki för WATCHDOG_TIMER



Figur 4. QSYS interface för WATCHDOG_TIMER_SYSTEM

5.2 Delsystem

Systemet innehålla följande komponenter:

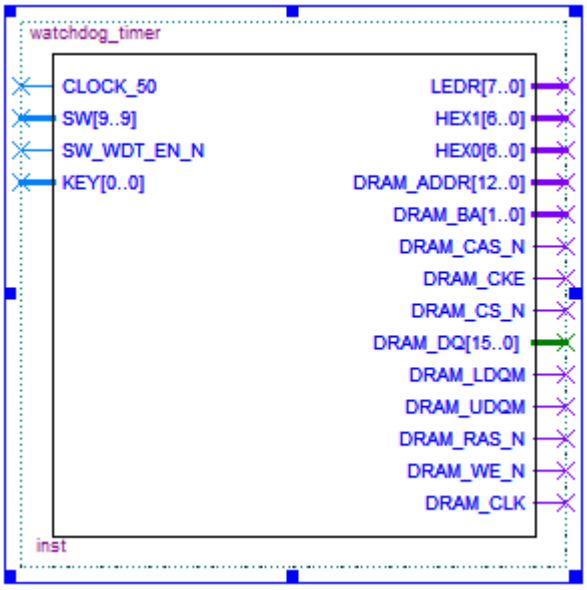
5.2.1 Toppnivån

Komponent namn: ***WATCHDOG_TIMER***

5.2.1.1 Systemarkitektur

Den toppnivån användas för att instansierar Qsys interface son innehålla hela IP komponenterna. Toppnivån får insignaler och skickar utsignaler till sjusegmentdisplayerna och lysdioder för att visar om watchdog timer är enable/disable.

TOPPNIVÅN symbol visas i Figur 5 och i Tabell 3 visar In- och utsignalerna.



Figur 5. TOPPNIVÅN för WATCHDOG_TIMER

Tabell 3. In- och utsignalerna till TOPP filen

Signal	**Namn**	**Riktning**	**Typ**
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	SW	in	std_logic
Switch enable/disable Watchdog timer	SW_WDT_EN_N	in	std_logic
Pio in_key	KEY	in	std_logic
Lysdioder	LEDR	out	std_logic_vector(7 downto 0)
7 Display segment	HEX1	out	std_logic_vector(6 downto 0)
7 Display segment	HEX0	out	std_logic_vector(6 downto 0)
External minne	SDRAM	out	std_logic, std_logic_vector

Ingången signaler
CLOCK_50: 50Mhz input clock.
sw(reset_n): Reset av signalen. Active low.
sw_wdt_en_n: Pin som enable/disable watchdog timer systemet.

Key: Om ”häng”-knappen trycks ner ska detta genererar ett feltillstånd som gör att C-programmet går in i en oändlig loop (hänger sig eller ”hang up”).

Utgången signaler
SDRAM: Externa minne som innehålla data och instruktioner.
ledr: När programmet kommer i feltillståndet ska 8 röda lysdioder tändas. Räkna hur många gånger det trycker på knappen.
hex0, hex1: Ändrar värde varje sekond.

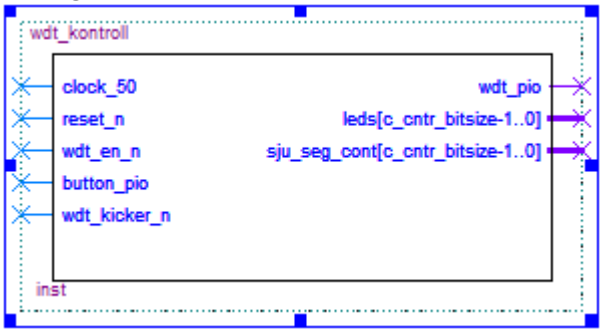
5.2.2 Watchdog Timer – komponent

Komponent namn: **wdt_KONTROLL**
Instansnamn: ***b0v_inst_wdt_kontroll***

5.2.2.1 Systemarkitektur

Genom denna komponent det kommer att hanterar hela processen av watchdog timer. Enable/disable watchdog timer, räkningen till sjusegmentsdisplayerna, lysdioder.

Wdt_KONTROLL-komponentens symbol visas i Figur 6 och i Tabell 4 visar In- och utsignalerna.



Figur 6. Interface av wdt_KONTROLL

Tabell 4. In- och utsignalerna till WATCHDOG_TIMER filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	RESET_N	in	std_logic
Enable/Disable watchdig timer	WDT_EN_N	in	std_logic
Pio in_key	BUTTON_PIO	In	std_logic
Kick_the_dog, återställas signaler	WDT_KICKER_N	In	std_logic

Hanterar på nios II feltillstånd	WDT_PIO	out	std_logic
Lysdioder	LEDS	out	std_logic_vector(7 downto 0)
7-segment display	SJU_SEG_CONT	out	std_logic_vector(7 downto 0)

Ingången signaler
CLOCK_50: *50Mhz input clock.*
reset_n: Reset av signalen. Active low.
wdt_en_n: sw(0), slå på och av "Watchdog_Timer".
button_pio: key_0 visar watchdog timer aktivitetet
wdt_kicker_n: "Kick_the_dog", återställas system.

Utgången signaler
wdt_pio: Hanterar på nios II feltillstånd.
leds: visar räkning och fel tillstånd.
sju_seg_cont: Räknaren för 7-segment display.

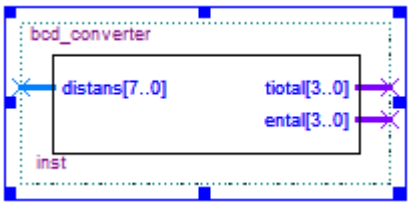
5.2.3 Bcd konverter - komponent

Komponent namn: ***bcd_converter***
*Instansnamn****: b1v_inst_bcd_converter***

5.2.3.1 Systemarkitektur

Denna komponent får avståndet i binär för att sen omvandlar i BCD för att kan visa på 7-segment display. Det använder metoden flyttar till vänster och adderar 3(shift-and-add-3).

BCD CONVERTER-komponentens symbol visas i Figur 7 och i Tabell 5 visar In- och utsignalerna.



Figur 7. Omvandlar binär till BCD

Tabell 5. In- och utsignalerna till BCD_CONVERTER filen

Signal	Namn	Riktning	Typ
Bits nummer	distans	in	std_logic_vector(7 downto 0)
Bcd om bas 10	tiotal	out	std_logic_vector(3 downto 0)
Bcd om bas 10	ental	out	std_logic_vector(3 downto 0)

Ingången signaler
distans: Nummer bits som kommer att omvandlas i bcd.

Utgången signaler
tiotal: bcd utgången som tiotal på bas 10.
ental: bcd utgången som ental på bas 10.

5.2.4 Sju_seg_displayer - komponent

Komponent namn: ***sju_displayer_komponent***
*Instansnamn****: b2v_inst_sju_displayer_komponent_1,***
b2v_inst_sju_displayer_komponent_2,

5.2.4.1 Systemarkitektur

Denna komponent styrprogram för 7-segment display för att visar avståndet.

SJU SEGMENT DISPLAYER-komponentens symbol visas i Figur 8 och i Tabell 6 visar In- och utsignalerna.



Figur 8. Sjusegmentdisplayer

Tabell 6. In- och utsignalerna till SJU_SEG_DISPLAYER filen

Signal	Namn	Riktning	Typ
Avstånd på bas 10	tal_in	in	std_logic_vector(3 downto 0)
Styrdata till HEX6	hex_out	out	std_logic_vector(6 downto 0)

Ingången signaler
tal_in: binärt tal (0-15).
Utgången signaler
hex_out: 7-segment display.

5.3 I/O pinnar

Figur 10 visar pinnar för DE10-Lite kortet.

Node Name	Direction	Location
altera_reserved_tck	Input	
altera_reserved_tdi	Input	
altera_reserved_tdo	Output	
altera_reserved_tms	Input	
CLOCK_50	Input	PIN_N14
DRAM_ADDR[12]	Output	PIN_R20
DRAM_ADDR[11]	Output	PIN_P20
DRAM_ADDR[10]	Output	PIN_T20
DRAM_ADDR[9]	Output	PIN_P19
DRAM_ADDR[8]	Output	PIN_P18
DRAM_ADDR[7]	Output	PIN_R18
DRAM_ADDR[6]	Output	PIN_T19
DRAM_ADDR[5]	Output	PIN_T18
DRAM_ADDR[4]	Output	PIN_U19
DRAM_ADDR[3]	Output	PIN_U18
DRAM_ADDR[2]	Output	PIN_V18
DRAM_ADDR[1]	Output	PIN_W19
DRAM_ADDR[0]	Output	PIN_U17
DRAM_BA[1]	Output	PIN_T22
DRAM_BA[0]	Output	PIN_T21
DRAM_CAS_N	Output	PIN_U21
DRAM_CKE	Output	PIN_N22
DRAM_CLK	Output	PIN_L14
DRAM_CS_N	Output	PIN_U20
DRAM_DQ[15]	Bidir	PIN_F22
DRAM_DQ[14]	Bidir	PIN_G19
DRAM_DQ[13]	Bidir	PIN_G20
DRAM_DQ[12]	Bidir	PIN_G22

Node Name	Direction	Location
DRAM_DQ[11]	Bidir	PIN_H22
DRAM_DQ[10]	Bidir	PIN_H21
DRAM_DQ[9]	Bidir	PIN_J22
DRAM_DQ[8]	Bidir	PIN_P21
DRAM_DQ[7]	Bidir	PIN_V21
DRAM_DQ[6]	Bidir	PIN_W20
DRAM_DQ[5]	Bidir	PIN_W22
DRAM_DQ[4]	Bidir	PIN_Y22
DRAM_DQ[3]	Bidir	PIN_AA21
DRAM_DQ[2]	Bidir	PIN_AA22
DRAM_DQ[1]	Bidir	PIN_Y20
DRAM_DQ[0]	Bidir	PIN_Y21
DRAM_LDQM	Output	PIN_V22
DRAM_RAS_N	Output	PIN_U22
DRAM_UDQM	Output	PIN_J21
DRAM_WE_N	Output	PIN_V20
HEX0[6]	Output	PIN_C17
HEX0[5]	Output	PIN_D17
HEX0[4]	Output	PIN_E16
HEX0[3]	Output	PIN_C16
HEX0[2]	Output	PIN_C15
HEX0[1]	Output	PIN_E15
HEX0[0]	Output	PIN_C14
HEX1[6]	Output	PIN_B17
HEX1[5]	Output	PIN_A18
HEX1[4]	Output	PIN_A17
HEX1[3]	Output	PIN_B16
HEX1[2]	Output	PIN_E18

HEX1[1]	Output	PIN_D18
HEX1[0]	Output	PIN_C18
KEY[0]	Input	PIN_B8
LEDR[7]	Output	PIN_D14
LEDR[6]	Output	PIN_E14
LEDR[5]	Output	PIN_C13
LEDR[4]	Output	PIN_D13
LEDR[3]	Output	PIN_B10
LEDR[2]	Output	PIN_A10
LEDR[1]	Output	PIN_A9
LEDR[0]	Output	PIN_A8
SW[9]	Input	PIN_F15
SW_WDT_EN_N	Input	PIN_C10

Figur 9. Pinnar för DE10-Lite kortet

6. VALIDERING KONSTRUKTIONEN

6.1 Testprotokoll för validering

I Tabell 7 visar testprotokoll för validering och länken visar en video for att validerats systemet.

Tabell 7. Testprotokoll för validering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Gör en reset(SW(9)) av systemet.	Reset CPU'n	Ja(se video)
2	Watchdog timer(SW(0))='1' enable	Systemet återställas och det fungerar normal.	Ja(se video)
3	SW(0)='1' enable, key(0)=trycks ner	Feltillstånd, återställas system	Ja(se video)

4	SW(0)='0' disable, key(0)=tryckner	Feltillstånd. Loop på systemet	Ja(se video)

https://www.youtube.com/watch?v=hdKtJ5Yam4A

7. MJUKVARAN ARKITEKTUR

7.1 Minneskarta

Memory-map från "sumary.html" för konstruktionen, se figur 10.

Nios II Memory Map

Slave Descriptor	Address Range	Size	Attributes
jtag_uart	0x08009068 - 0x0800906F	8	printable
sysid	0x08009060 - 0x08009067	8	
TIMER_HW_IP_0	0x08009050 - 0x0800905F	16	
pll_0	0x08009040 - 0x0800904F	16	
wdt_KICKER_N	0x08009030 - 0x0800903F	16	
wdt_PIO	0x08009020 - 0x0800902F	16	
button_pio	0x08009010 - 0x0800901F	16	
wdt_LEDR	0x08009000 - 0x0800900F	16	
onchip_ram	0x08004000 - 0x08007FFF	16384	memory
new_sdram_controller_0	0x04000000 - 0x07FFFFFF	67108864	memory

Figur 10. Flödesdiagram

7.2 Drivrutiner

Det visar en översikt över de funktioner som krävdes av kravspecifikationen.

7.2.1 Timer-drivrutiner

För att använda timer-drivrutinen finns de macron som är listade i Tabell 8.

Tabell 8. Timmer-drivrutiner i altera_avalon_timer_regs.h

Funktion	Beskrivning
#define TIMER_STOP IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x00000000)	Stoppar timern
#define TIMER_RESET IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x40000000)	Nollställer timern
#define TIMER_START IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x80000000)	Startar timern
#define TIMER_READ IORD_32DIRECT(TIMER_HW_IP_0_BASE,0)	Läser ut timervärdet

7.3 C koden

I figur 11 visar c-koden för att validerar WATCHDOG_TIMER komponent på mjukvaran.

```
#include <system.h>
#include <alt_types.h>
#include <stdio.h>
#include <altera_avalon_pio_regs.h>
//#include <altera_avalon_timer_regs.h> //device driver for timer ip

//int value_hertz = 50000000;
//void delay_hertz_second(int *param_hertz);    //device driver for ändra ändra samplingsfrekvens

void kicker_wdt(int wdt_pio);
void kontroll_leds(int leds);

int main(void) {
    int buttonpio=0;
    int in_wdt_pio=0;

    printf("We start.");
    kontroll_leds(0x0); //clear leds

    while (1) {
        /* Den här delay det funkar inte för att visa sju_segment_display,
         * jag har skapat en delay på hardware istället */
        //delay_hertz_second(&value_hertz);

        kicker_wdt(0x1); // kick_the_dog
        buttonpio = 0x1 & IORD_ALTERA_AVALON_PIO_DATA(BUTTON_PIO_BASE); //laser key_0
        if (buttonpio==0) {
            /* läser om watchdog timer (SW[0]) är enable/disable */
            in_wdt_pio = IORD_ALTERA_AVALON_PIO_DATA(WDT_PIO_BASE);

            /* Feltillstånd. Watchdog_timer är inaktiv ska det inte påverka systemet.
               knappen_0 trycks ner och det kommer att starta om kounter
               SW[0]='1' är inaktiv."Watchdog_Timer" inte påverka systemet.*/
            kontroll_leds(0xff);    //tänder 8 lysdioder
            kicker_wdt(0x0);        //systemet återställas
            while (buttonpio == 0x0 && in_wdt_pio == 0x1){
                /* buttonpio(key_0) är släpps = 1*/
                buttonpio = 0x01 & IORD_ALTERA_AVALON_PIO_DATA(BUTTON_PIO_BASE);
            }

            /* Feltillstånd. Watchdog_timer är aktiv ska det påverka systemet.
                knappen_0 trycks ner och programmet komma in loopen
                LOOP. SW[0]='0' är aktiv."Watchdog_Timer" påverka systemet.*/
            while (buttonpio == 0x0 && in_wdt_pio == 0x0){
                /* Programmet väntar att komma ur loopen om processorn resettas
                 * SW[0]='1' */
                in_wdt_pio = IORD_ALTERA_AVALON_PIO_DATA(WDT_PIO_BASE);
            }
            kontroll_leds(0x0);
        }
    }
    return 0;
}

void kicker_wdt(int wdt_pio){
    IOWR_ALTERA_AVALON_PIO_DATA(WDT_KICKER_N_BASE, wdt_pio); // kick_the_dog
}

void kontroll_leds(int leds){
    if (leds == 0xff){
        IOWR_ALTERA_AVALON_PIO_DATA(WDT_LEDR_BASE, 0xff);
    }else{IOWR_ALTERA_AVALON_PIO_DATA(WDT_LEDR_BASE, 0x0);}
}
```

Figur 11. WATCHDOG_TIMER validering